



UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

FACULTAD DE INGENIERÍA

PROGRAMA ACADÉMICO DE CIENCIAS DE LA COMPUTACIÓN

TRABAJO PRÁCTICO — SEMANA 6

**Arquitecturas CNN: Comparación, Batch Normalization y Transfer
Learning**

Integrantes

Quispe Palacin, Diego Eloy (u202012453)

PROFESOR

Pinedo Taquia, Jairo

Lima, 17 de mayo de 2026

1. Introducción

El diagnóstico médico asistido por inteligencia artificial ha cobrado gran relevancia en los últimos años, especialmente en el análisis de imágenes clínicas. La clasificación automática de células sanguíneas es un problema de visión computacional con aplicación directa en hematología, donde la identificación precisa de tipos celulares permite detectar enfermedades como leucemia, infecciones o anemias.

En este trabajo se implementaron, entrenaron y compararon arquitecturas clásicas de redes neuronales convolucionales sobre el Blood Cell Image Dataset, un dataset público de imágenes microscópicas con 4 clases: EOSINOPHIL, LYMPHOCYTE, MONOCYTE y NEUTROPHIL. El objetivo no fue solo obtener buena precisión, sino comprender empíricamente el impacto de cada componente y decisión de diseño, incluyendo la arquitectura, el uso de Batch Normalization y el Transfer Learning.

2. Metodología

2.1. Dataset

Se utilizó el Blood Cell Image Dataset disponible en Kaggle. El dataset contiene aproximadamente 12,500 imágenes RGB de 320x240 píxeles, divididas en 4 clases: EOSINOPHIL, LYMPHOCYTE, MONOCYTE y NEUTROPHIL. El split train/test fue el provisto por el dataset, con aproximadamente 9,957 imágenes de entrenamiento y 2,487 de test. Las clases están balanceadas, por lo que no se requirieron técnicas de rebalanceo.

2.2. Preprocesamiento

Todas las imágenes fueron redimensionadas a 64×64 píxeles y convertidas a tensor. Se aplicó normalización con media [0.485, 0.456, 0.406] y desviación estándar [0.229, 0.224, 0.225], correspondientes a los valores estándar de ImageNet.

2.3. Arquitecturas implementadas

Se implementaron desde cero LeNet-5 adaptado y VGG-11 simplificado, cada uno con y sin Batch Normalization. LeNet-5 fue adaptado a entrada 64×64×3 manteniendo su estructura original de 2 bloques conv-pool y 3 capas FC. VGG-11 fue simplificado reduciendo a la mitad los filtros de cada bloque.

2.4. Hiperparámetros

- Optimizador: Adam
- Learning rate: 0.0001
- Batch size: 32
- Épocas: 40 (modelos desde cero), 20 (Transfer Learning)
- Función de pérdida: CrossEntropyLoss
- Semilla: fijada para reproducibilidad

2.5. Transfer Learning

Se utilizó VGG-16 preentrenada en ImageNet. Se compararon tres estrategias: Feature Extraction (solo FC final entrenable), Fine-tuning parcial (primeros 2 bloques congelados) y Fine-tuning total (todo el modelo entrenable con lr=0.00001).

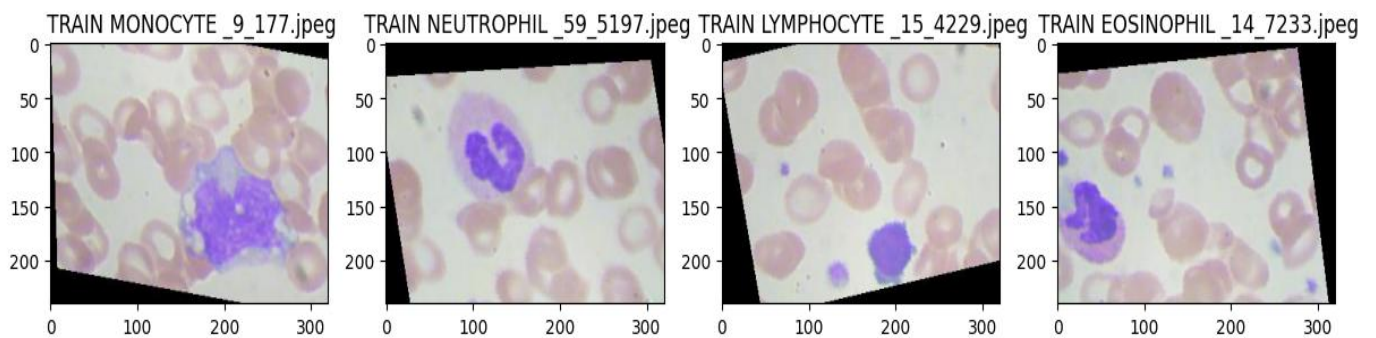
3. Resultados

3.1. Análisis Exploratorio del Dataset

El dataset presenta imágenes microscópicas con características morfológicas y de coloración distintivas por clase:

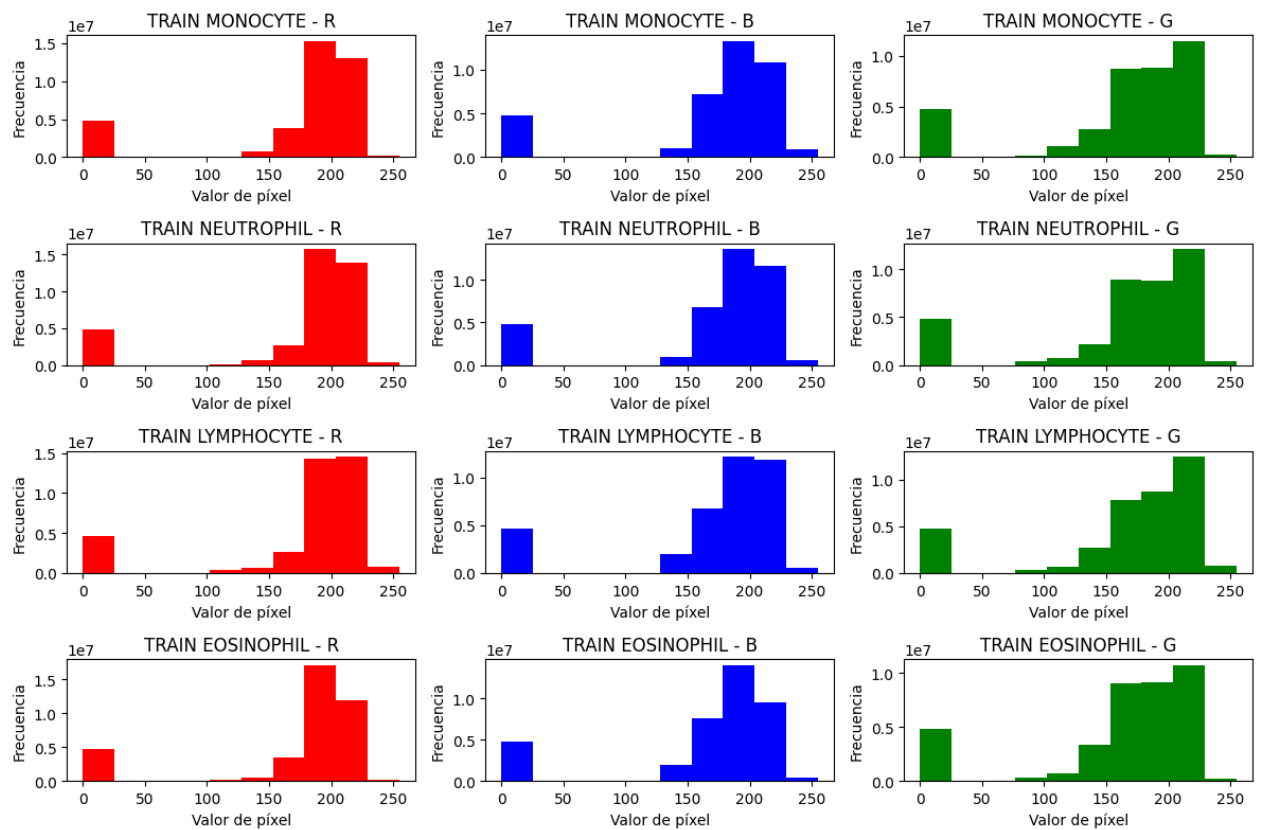
- **MONOCYTE**: células de tamaño mayor al promedio, con coloración más tenue respecto al resto.
- **NEUTROPHIL**: presentan múltiples lóbulos internos visibles, con coloración más intensa.
- **LYMPHOCYTE**: célula única sin capa exterior visible, con coloración muy intensa y concentrada.
- **EOSINOPHIL**: presentan separaciones internas, pero más dispersas, con coloración morada menos intensa.

Figura 1 Células sanguíneas por clase



El análisis de histogramas RGB mostró que las distribuciones de intensidad de píxeles son similares entre clases, con alta superposición en valores altos de intensidad. Las diferencias más notorias se encontraron en los canales R y B para EOSINOPHIL, y en el canal G para LYMPHOCYTE. Esto sugiere que el color por sí solo no es suficiente para distinguir las clases. El modelo tendrá que aprender características morfológicas como forma y textura.

Figura 2 Histogramas de intensidad RGB por clase



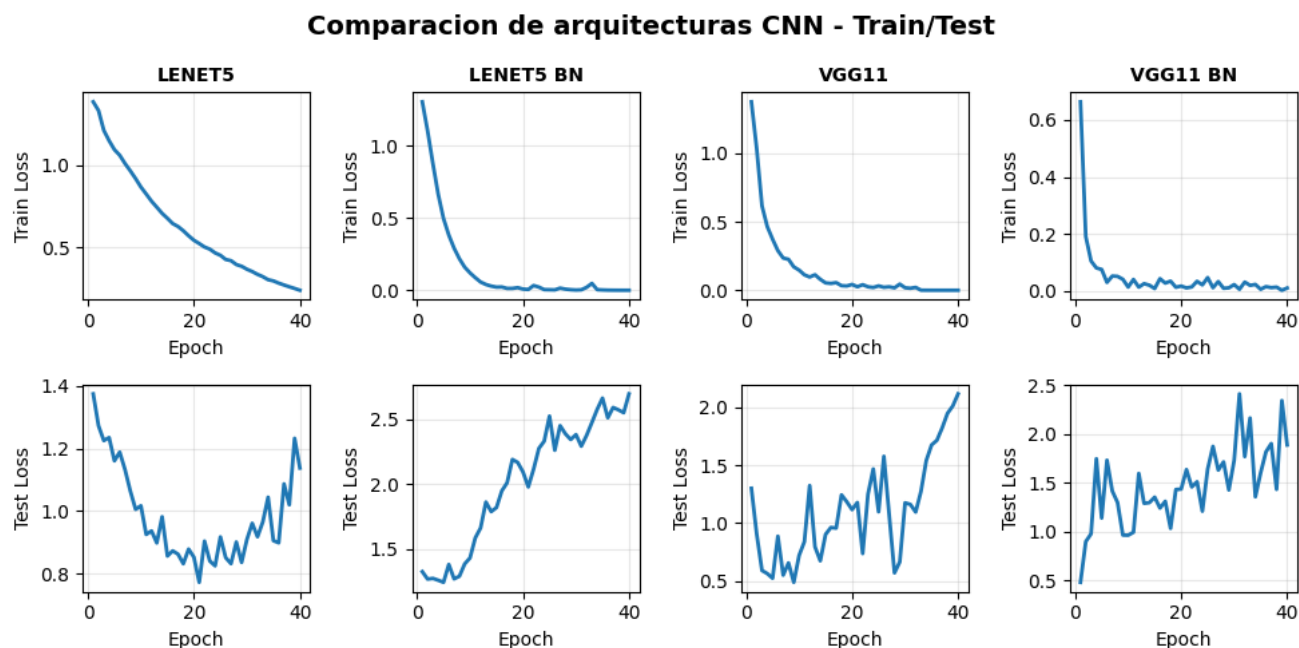
Se verificó que todas las imágenes tienen dimensiones 320×240 píxeles y ninguna imagen corrupta fue encontrada en el dataset.

3.2. Comparación de arquitecturas CNN

Se entrenaron 4 variantes durante 40 épocas con learning rate de 0.0001 y batch size de 32.

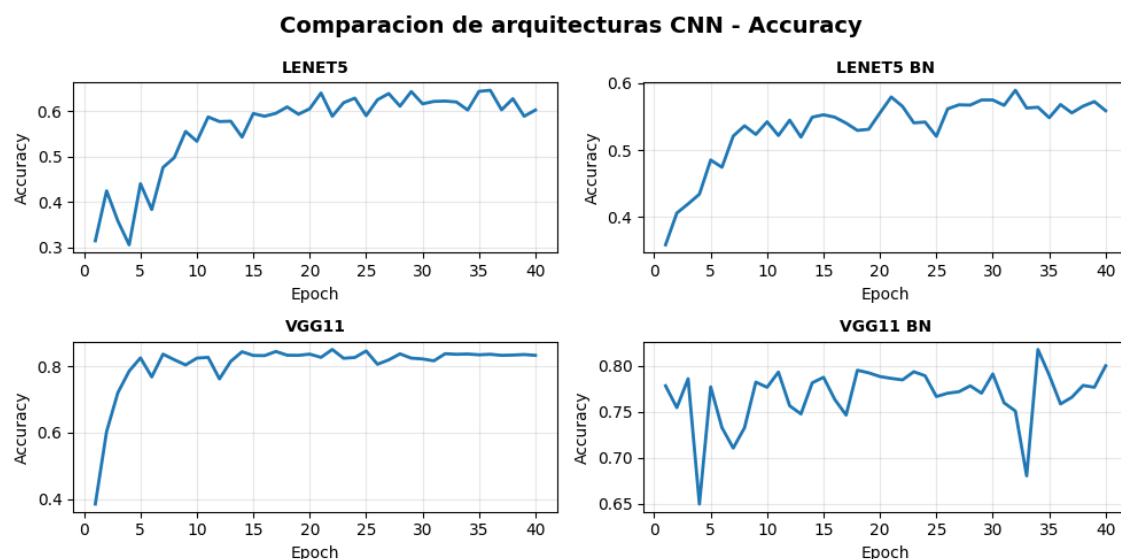
Modelo	Parámetros	Tiempo/época	Mejor Accuracy
LeNet5	337,976	~12s	64.6%
LeNet5 BN	338,020	~12s	58.9%
VGG11	8,609,988	~13s	85.1%
VGG11 BN	8,612,740	~13s	81.7%

Figura 4 Curvas de Train Loss y Test Loss por arquitectura CNN



En la comparación de curvas train/test, todos los modelos generan un train loss bajo con convergencia similar, exceptuando LeNet5 que converge más lento. Sin embargo, en el test loss la situación cambia, pues todos presentan overfitting. El menor es LeNet5 con un overfitting moderado, mientras que los demás, especialmente LeNet5 BN, presentan overfitting alto.

Figura 3 Curvas de Accuracy por arquitectura CNN



En cuanto al accuracy, VGG11 confirma ser el mejor modelo con 85.1%, reafirmando que una arquitectura más profunda captura mejor las características de las células.

3.3. Transfer Learning

Se utilizó VGG16 preentrenada en ImageNet comparando tres estrategias durante 20 épocas.

Modelo	Parámetros	Tiempo/época	Mejor Accuracy
VGG16 Feature Extraction	134,276,932	~27s	40.5%
VGG16 Fine-tuning parcial	134,276,932	~32s	85.0%
VGG16 Fine-tuning total	134,276,932	~35s	80.7%

Figura 5 Curvas de Train Loss y Test Loss para estrategias de Transfer Learning

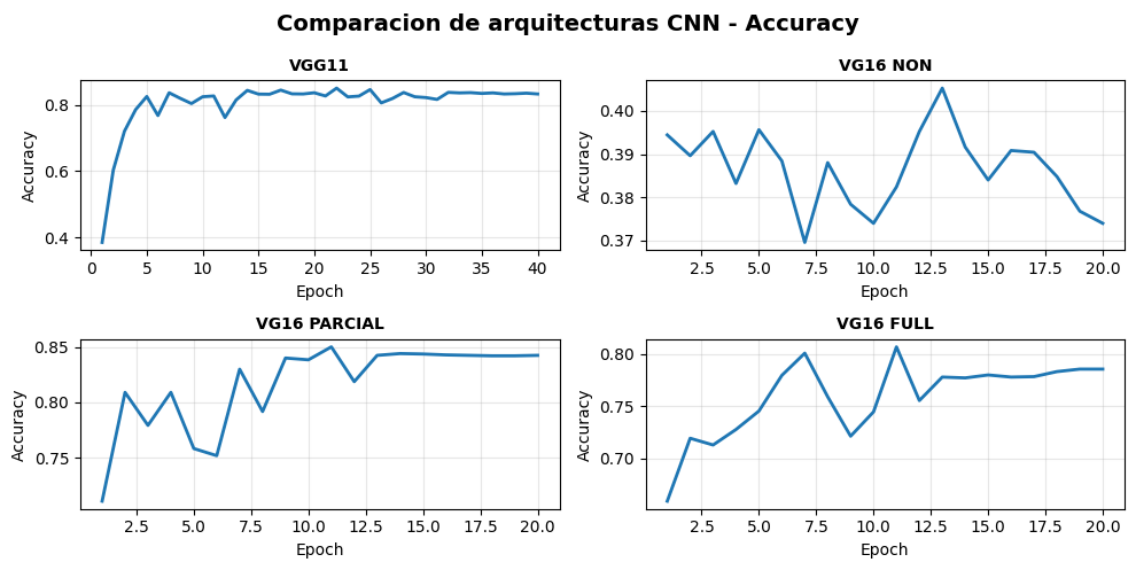
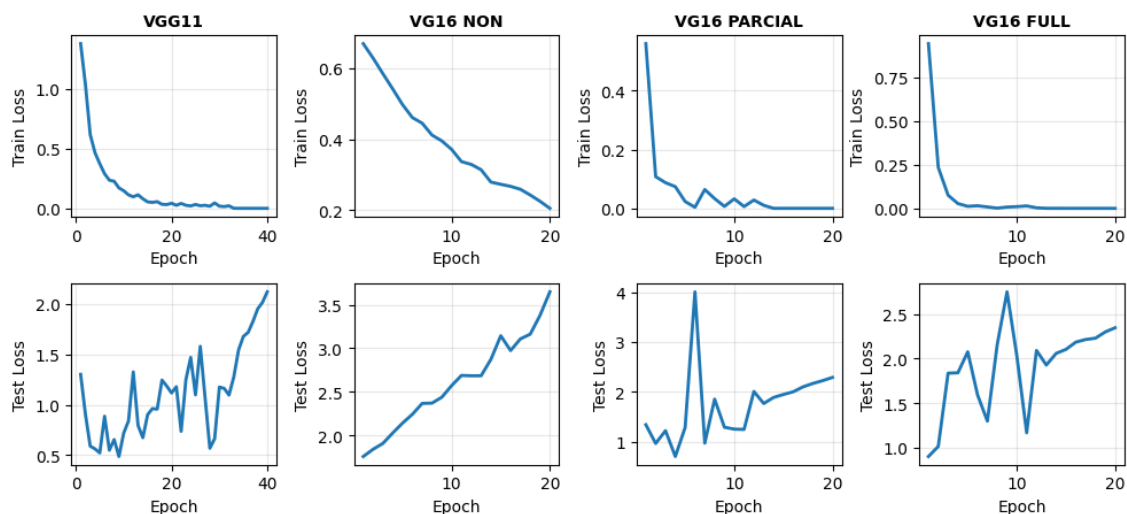


Figura 6 Curvas de Accuracy para estrategias de Transfer Learning



Para VGG16 Feature Extraction, con solo el classifier habilitado, no se obtuvo una buena convergencia respecto a los demás modelos. En el test loss se nota nuevamente overfitting, aunque comparado a los otros modelos tiene un comportamiento más estable. Es el peor en accuracy con un valor inestable y el más bajo de todos, pues al tener todo congelado hace que el aprendizaje sea prácticamente nulo.

VGG16 Fine-tuning parcial resultó el mejor. Al mantener los primeros bloques congelados, el modelo no necesita mover todos sus parámetros sino solo una parte, lo que facilita la adaptación al nuevo dominio. Además, esto genera mayor eficiencia en el tiempo de entrenamiento.

VGG16 Fine-tuning total, aunque similar al parcial, presentó mayor overfitting pero con una convergencia más estable. Al habilitar todo el modelo con un lr muy pequeño, la adaptación es más lenta pero más controlada.

3.4. Convergencia hacia el 80% de accuracy

Modelo	Épocas para 80%	Mejor Accuracy
LeNet5	Nunca llegó	64.6%
LeNet5 BN	Nunca llegó	58.9%
VGG11	5	85.1%
VGG11 BN	34	81.7%

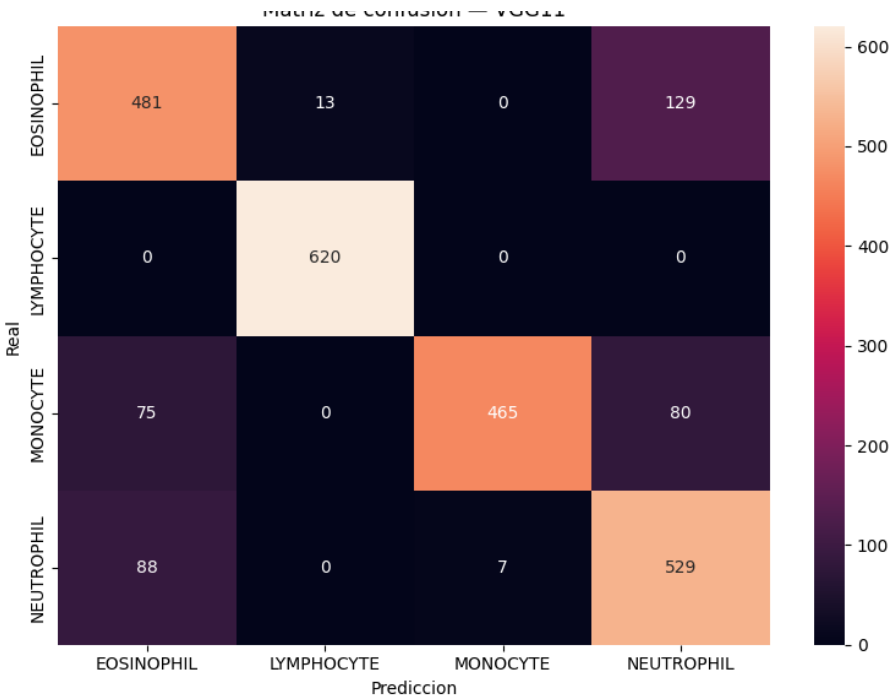
VGG11 llegó al 80% en tan solo 5 épocas, mientras que VGG11 BN necesitó 34. Los modelos LeNet5 y LeNet5 BN no llegaron al 80% en las 40 épocas entrenadas. Esto reafirma que VGG11 es el modelo más eficiente en convergencia de todos los evaluados.

Contrario a lo esperado, Batch Normalization ralentizó la convergencia en lugar de acelerarla. Esto se debe al learning rate usado, pues BN suaviza el terreno donde se mueve el error, por lo que los pasos dados son importantes. Con $lr=0.0001$ los pasos son tan pequeños que el modelo avanza muy lento. Se estima que un lr óptimo para modelos con BN podría estar entre 0.0001 y 0.001, como sería 0.0005, aunque esto no fue probado experimentalmente. Se puede suponer que con el lr correcto, VGG11 BN podría superar a VGG11.

3.5. Matrices de confusión

Se desarrollaron 2 matrices de confusión para los mejores modelos de cada categoría.

Figura 7 Matriz de confusión del mejor modelo desde cero - VGG11



El primer para VGG11, mejor modelo desde cero. Se puede ver que el modelo tiene una predicción aceptable, aunque en la clase EOSINOPHIL tiene más dificultades, confundiéndose principalmente con NEUTROPHIL. En las clases MONOCYTE y NEUTROPHIL el error es menor, y en LYMPHOCYTE la clasificación es perfecta con 620/620.

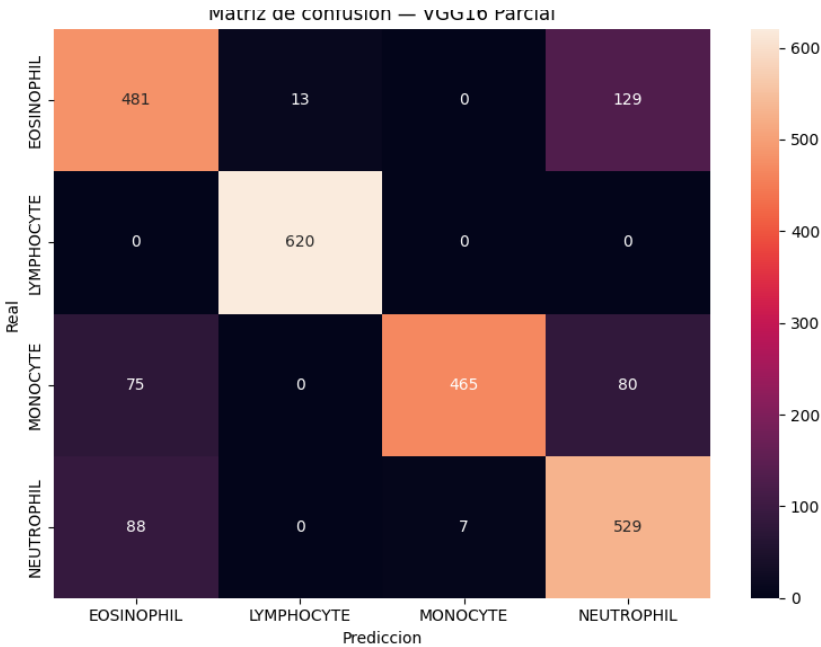
El segundo para VGG16 Fine-tuning parcial, mejor modelo de Transfer Learning. El patrón de errores es prácticamente idéntico al anterior, lo que indica que ambos modelos, a pesar de tener arquitecturas y estrategias de entrenamiento distintas, aprendieron patrones similares del dataset y cometen los mismos tipos de errores.

4. Discusión

4.1. Impacto de la profundidad de la arquitectura

VGG11 superó ampliamente a LeNet5 con 85.1% vs 64.6%. LeNet5 fue diseñada originalmente para dígitos en escala de grises, por lo que su capacidad es insuficiente para distinguir patrones complejos como la forma de los lóbulos en NEUTROPHIL o la textura de EOSINOPHIL. VGG11 al ser más profunda, puede aprender representaciones más ricas y abstractas de las células.

Figura 8 Matriz de confusión del mejor modelo de Transfer Learning, VGG16 Fine-tuning parcial



4.2. Efecto de Batch Normalization

Batch Normalization no mejoró el rendimiento en este experimento. Según Ioffe & Szegedy (2015), BN reduce el Internal Covariate Shift normalizando las activaciones entre capas, permitiendo usar learning rates más altos y acelerando la convergencia. Sin embargo, el uso de un lr bajo para todos los modelos anuló esta ventaja. Se considera que con un lr óptimo, los modelos con BN podrían superar a sus contrapartes sin BN.

4.3. Transfer Learning

VGG16 Fine-tuning parcial igualó al mejor modelo desde cero con 85% en menos épocas, demostrando la utilidad de los pesos preentrenados. Al mantener congelados los primeros bloques se conservan features generales como bordes y texturas, adaptando solo las capas superiores a lo nuevo. Esta estrategia es especialmente recomendada en contextos de datos médicos limitados, donde reentrenar todo el modelo puede llevar al sobreajuste.

4.4. Limitaciones y propuestas de mejora

- No se aplicaron técnicas de regularización como Dropout ni Data Augmentation, lo cual generó overfitting en todos los modelos. Incorporar estas técnicas sería la mejora más importante.
- No se realizó búsqueda exhaustiva de hiperparámetros, especialmente del learning rate para modelos con BN. Un lr entre 0.0001 y 0.001 podría mejorar significativamente los resultados. Por ejemplo, $lr = 0.0005$
- Los modelos de Transfer Learning fueron entrenados con solo 20 épocas, lo que puede haber limitado su rendimiento final. Se estima 50 épocas para mejores resultados.

5. Conclusiones

En este trabajo se implementaron, entrenaron y compararon arquitecturas clásicas de redes neuronales convolucionales para la clasificación de células sanguíneas. Los principales hallazgos fueron:

- VGG11 resultó el mejor modelo desde cero con 85.1% de accuracy, confirmando que arquitecturas más profundas capturan mejor las características de las células.

- Batch Normalization no mejoró el rendimiento en este experimento debido al uso de un learning rate inadecuado. Se estima que un lr óptimo entre 0.0001 y 0.001 podría revertir este resultado.
- Transfer Learning con Fine-tuning parcial igualó al mejor modelo desde cero en menos épocas, siendo la estrategia recomendada para contextos de datos médicos limitados. Otra estrategia más, sería el uso de Data Augmentation por mejorar la variedad de los datos para el aprendizaje.
- El overfitting fue una limitación presente en todos los modelos, atribuible a la ausencia de Dropout y Data Augmentation.
- La clase EOSINOPHIL fue la más difícil de clasificar en todos los modelos, confundiéndose principalmente con NEUTROPHIL, posiblemente por sus similitudes.

6. Referencias

- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS*.
- Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *ICLR 2015*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *CVPR*.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *ICML*.
- Mooney, P. (2018). Blood Cell Images. Kaggle. <https://www.kaggle.com/datasets/paultimothymooney/blood-cells>